Student Assignment 1 — HotpotQA: From RAG to Multi-Agent Investigation

Duration: 4–5 days | **Focus:** IR, RAG, Agentic RAG, multi-hop investigation, verification

Scenario: Build an AI Investigation Assistant that answers complex questions using evidence from multiple documents.

Research question: When does iterative or multi-agent investigation provide enough benefit over conventional RAG to justify the extra cost and complexity?

> **Mandatory working method:** Build → Evaluate → Analyze failures → Form a hypothesis → Improve → Re-evaluate → Compare. Do not jump directly to agents. For every cycle explain: what you built, why, pros/cons, metrics, failure examples, what motivates the next cycle, and whether the change actually helped.

Compute & scope

This project is designed for a standard laptop/CPU or limited GPU. Do **not** train or fine-tune a large LLM. Use a representative subset.

Activity	Suggested size
EDA / development pool	1,000–3,000 examples
Retrieval evaluation	500–1,000
End-to-end RAG evaluation	200–500
Single-agent evaluation	100–300
Multi-agent evaluation	100–300
Manual error analysis	20–30

If compute/time is limited, reduce sample size rather than removing evaluation and failure analysis.

Mandatory final comparison

Metric	Baseline	Improved pipeline	Single agent	Multi-agent
Main quality metric				
Evidence / retrieval quality				
Unsupported answers / errors				
Avg retrieval calls				
Avg LLM calls				
Tokens / estimated cost				
Latency				
Complexity / debuggability				

Final presentation

15–20 minutes: Problem → Dataset → Baseline → Evaluation → Failure analysis → Improved pipeline → Single agent → Multi-agent → Comparison → What you would deploy and why.

Required: show one baseline failure fixed later, one remaining failure, one case where an agent helped, one where it was unnecessary, and one case where multi-agent added little or failed.

What I would NOT use: identify at least two technologies/approaches you deliberately rejected and explain why.

Cycle 0 — Understand the data

Inspect questions, answers, context and supporting facts. Show 3–5 examples and explicitly map the evidence chain. Explain multi-hop QA and why it is harder than ordinary QA.

Cycle 1 — Basic RAG

```
Question → Retriever → Top-K evidence → LLM → Answer
```

Choose and justify embedding/retrieval, chunking and Top-K. Evaluate answer EM/F1, Evidence Recall@K, latency and calls. For errors separate **retrieval failure** from **reasoning/generation failure**.

Cycle 2 — Improved non-agent RAG

Based on Cycle 1 failures, test a justified improvement: BM25, dense, hybrid retrieval, query transformation, reranking, chunking or Top-K. Do not add techniques without a hypothesis.

```
Question → BM25 + Dense → candidates → Reranker → Top-K → LLM
```

Be ready to explain BM25 vs embeddings, lexical vs semantic retrieval, hybrid fusion, bi-encoder vs cross-encoder and latency/quality trade-offs.

Cycle 3 — Single Agent: Agentic RAG

```
Question → Investigation Agent → Search → Observe → Enough evidence?
                                                  ├─ Yes → Answer
                                                  └─ No  → identify missing info → new query → Search ...
```

Use a deterministic safety limit such as 3–5 steps. Measure whether iterative retrieval helps multi-hop cases and whether the agent over-searches, stops early or loops.

> **Key defense:** It is agentic because the next retrieval action/query depends on intermediate evidence observed at runtime.

Cycle 4 — Multi-Agent Investigation

```
Question → Planner → Researcher → Evidence → Critic/Verifier
                   ↑                          │
                   └──── missing evidence? ───┘
                                ↓ sufficient
                          Synthesizer → Answer
```

Define distinct responsibilities. Hypothesis example: independent verification reduces premature stopping and unsupported answers. Compare against the single agent; multi-agent is mandatory to test, not mandatory to win.

Questions you must be able to answer

- Why RAG? Why not direct LLM?
- Why this retriever and Top-K?
- When does BM25 beat dense retrieval?
- Why rerank?
- Why did improved RAG still fail?
- What exactly makes the system an agent?
- Why multi-agent instead of one agent with multiple tools?
- How do you stop loops and debug decisions?
- When would you deploy normal RAG instead?